Dispositivos Conectados

Projeto Final

2026/2027

Versão 1.0
17 de outubro de 2026

Histórico de versões

Versão	Data	Alterações
1.0	2026-10-17	Versão inicial.

1 Introdução

Os sistemas *embedded* estão presentes em praticamente todos os domínios tecnológicos, desde automação industrial, telecomunicações e dispositivos médicos, até sistemas de transporte, energia e Internet das Coisas (IoT). A sua importância advém da capacidade de integrar *hardware* e *software* num único sistema dedicado, capaz de executar tarefas específicas com elevada eficiência, fiabilidade e baixo consumo energético.

Num contexto em que a conectividade, a recolha de dados e a autonomia dos dispositivos assumem um papel central, os sistemas *embedded* representam o alicerce da inovação digital. São eles que permitem transformar sensores e atuadores em sistemas inteligentes, capazes de comunicar, aprender e responder em tempo real. O domínio desta área exige compreender não apenas os fundamentos da eletrónica e da programação de microcontroladores, mas também as arquiteturas de comunicação, os protocolos de rede e as estratégias de otimização de energia.

Em contexto industrial é frequente que o cliente reveja os requisitos durante o desenvolvimento, por vezes invalidando decisões de arquitetura já tomadas. Absorver essa mudança sem reescrever o sistema de raiz constitui uma competência de engenharia tão relevante como a de o construir da primeira vez, e é raramente exercitada em contexto académico.

> **Nota ao grupo:** o cliente reserva-se o direito de rever a especificação durante o período de desenvolvimento. Qualquer revisão será comunicada através do *eLearning* e passará a integrar o âmbito do projeto. As decisões de arquitetura devem ser tomadas tendo em conta esta possibilidade.

2 Projeto

O projeto consiste na conceção e implementação do **EcoNode**, um nó de monitorização ambiental autónomo baseado no microcontrolador ESP32-C6 DevKitC-1, destinado a locais sem infraestrutura de rede fiável (estufas, arquivos, armazéns, salas técnicas).

O dispositivo recolhe dados de temperatura e humidade ambiente através de um sensor DHT20, apresenta informação num ecrã TFT e utiliza um LED RGB para indicar o estado do sistema. Os dados recolhidos são registados num cartão microSD para posterior análise e publicados via Wi-Fi num servidor de telemetria remoto, permitindo a sua visualização em tempo real. Adicionalmente, um *log* de operações é registado no cartão microSD, estabelecendo um mecanismo de auditoria ao funcionamento do dispositivo. O controlo do brilho do ecrã é concretizado através de um potenciómetro analógico, garantindo uma interface ajustável ao utilizador e contribuindo para otimizar o consumo energético.

O EcoNode expõe ainda uma **interface Bluetooth Low Energy**, destinada a dois usos: consultar as leituras localmente a partir de um telemóvel, mesmo quando não existe cobertura Wi-Fi, e provisionar e configurar o dispositivo em campo, sem necessidade de retirar o cartão microSD nem de aceder fisicamente ao equipamento.

A configuração do sistema (periodicidade de amostragem, credenciais de rede, endereço do servidor de telemetria, tópicos ou recursos de publicação, entre outros parâmetros) é feita utilizando um ficheiro JSON armazenado no cartão microSD. Desta forma, é possível ajustar o comportamento do dispositivo sem necessidade de recompilar ou reprogramar o *firmware*, aumentando a flexibilidade e facilitando a manutenção.

2.1 Requisitos

O projeto desenvolvido deverá cumprir o conjunto de requisitos funcionais e não funcionais especificado em seguida. Estes requisitos, juntamente com os *aspetos de implementação a observar*, descrevem globalmente a solução pretendida pelo cliente.

2.1.1 Funcionais

Os requisitos funcionais definem as operações que o dispositivo deve executar, ou seja, as funções concretas que o sistema deve cumprir para atingir os objetivos propostos. Para este projeto, estabelecem-se os seguintes requisitos funcionais:

- **RF1.** Leitura da temperatura e humidade com período configurável [1 s - 3600 s].
- **RF2.** Visualização de dados no ecrã TFT, incluindo os valores medidos, o estado da ligação de rede e o estado da interface BLE.
- **RF3.** Indicação de estado através do LED RGB [normal: verde | erro: vermelho | transmissão: amarelo | provisionamento: azul].
- **RF4.** Registo das leituras e do *log* de operações em ficheiros no cartão microSD.
- **RF5.** Publicação das leituras de temperatura e humidade num servidor de telemetria remoto, com visualização em tempo real.
- **RF6.** Controlo de brilho do ecrã através de potenciómetro.
- **RF7.** Leitura, interpretação e *enforcing* de configurações a partir de ficheiro JSON armazenado no cartão microSD.

- **RF8.** Exposição das leituras através de um *GATT server* BLE, implementando o *Environmental Sensing Service* (UUID 0x181A) com as características *Temperature* (0x2A6E) e *Humidity* (0x2A6F), em modo de leitura e notificação.
- **RF9.** Exposição de um serviço GATT proprietário de configuração, que permita consultar e alterar, no mínimo, a periodicidade de amostragem, as credenciais da rede sem fios e o endereço do servidor de telemetria.
- **RF10.** Provisionamento em campo: na ausência de configuração de rede válida, o dispositivo entra em modo de provisionamento e aceita a configuração por BLE, persistindo-a no cartão microSD.
- **RF11.** Implementação e documentação de uma estratégia de redução de consumo energético.

2.1.2 Não funcionais

Os requisitos não funcionais definem como o sistema deve funcionar sem descrever funcionalidades específicas. Para este projeto, estabelecem-se os seguintes requisitos não-funcionais:

- **RNF1.** Código modular e documentado. As diferentes funcionalidades devem ser distribuídas por tarefas FreeRTOS individuais. Nenhuma tarefa deve bloquear a execução de outra.
- **RNF2.** Tolerância a falhas de rede. A perda de conectividade Wi-Fi ou do servidor de telemetria não pode interromper a amostragem, o registo em cartão nem a interface BLE.
- **RNF3.** Tempo de resposta adequado ao contexto de monitorização.
- **RNF4.** Interface simples e intuitiva.
- **RNF5.** Alteração de parâmetros de funcionamento sem necessidade de recompilação.
- **RNF6.** Adoção de práticas de segurança robustas. Em particular, credenciais de rede não podem ser apresentadas no ecrã nem escritas no ficheiro de *log*, e as características BLE que as recebem não podem ser legíveis.
- **RNF7.** Cada módulo deve expor uma interface pública (`.h`) bem definida. Módulos consumidores de dados não devem depender uns dos outros: a título de exemplo, o módulo de visualização não deve incluir cabeçalhos do módulo de rede.

2.1.3 Aspetos de implementação a observar

Ambiente de desenvolvimento. O projeto deve compilar com o **ESP-IDF v6.1.0**, versão recomendada para o laboratório. Deve ser possível descomprimir o repositório e executar o *build* em qualquer máquina com essa versão instalada, sem passos manuais adicionais. Em particular, todas as dependências externas devem estar declaradas no ficheiro `idf_component.yml` do projeto.

Note-se que, a partir da versão 6.0 do ESP-IDF, vários componentes deixaram de fazer parte da árvore principal e passaram a ser obtidos através do gestor de componentes. É o caso do cliente MQTT (`espressif/mqtt`) e da biblioteca de manipulação de JSON (`espressif/cjson`). A ausência destas declarações impede a compilação do projeto noutra máquina.

Comunicação da tarefa de aquisição. A implementação da solução deve usar uma tarefa FreeRTOS específica para a leitura da temperatura e humidade relativa com a periodicidade definida. Esta periodicidade deve ser configurada no arranque do sistema, lendo o ficheiro de configuração JSON. A amostra adquirida, incluindo valores medidos e metadados associados, deve ser entregue ao resto do sistema por um dos seguintes mecanismos:

A) *Queue*

B) *Message Buffer*

O mecanismo a empregar por cada grupo será sorteado em aula e só é exigido **para esta tarefa específica**. As restantes tarefas podem usar qualquer método de comunicação que seja considerado adequado.

Em qualquer dos casos, a documentação deve justificar o dimensionamento adotado (profundidade da fila ou dimensão do *buffer*) e descrever o comportamento do sistema na situação em que a capacidade se esgota. A solução adotada para essa situação será objeto de discussão na defesa.

Parâmetros atribuídos ao grupo. Alguns parâmetros do sistema são específicos de cada grupo e derivam dos números mecanográficos dos seus elementos. Seja S a soma de todos os algarismos que compõem os números mecanográficos dos dois estudantes, e seja $D = S \bmod 10$.

- **Identificador do nó** (`node_id`): $1000 + 10 \times T + D$, onde T é o número da turma prática.
- **Base dos tópicos ou recursos de telemetria**: `econode/<node_id>/`.
- ***UUID* base do serviço GATT proprietário de configuração**: `E0C0DE00-0000-4000-8000-00000000XXXX`, em que `XXXX` é o `node_id` representado em quatro dígitos hexadecimais. As características do serviço usam o mesmo UUID com o primeiro grupo alterado para `E0C0DE01`, `E0C0DE02` e assim sucessivamente.

Exemplo: turma P2, estudantes com NMEC 112233 e 445566. $S = (1+1+2+2+3+3) + (4+4+5+5+6+6) = 42$, logo $D = 2$. O identificador do nó é $1000 + 10 \times 2 + 2 = 1022$, os tópicos têm a base `econode/1022/` e o *UUID* base do serviço é `E0C0DE00-0000-4000-8000-0000000003FE`.

Estes valores devem constar do ficheiro de configuração e ser apresentados no ecrã TFT durante o arranque.

Transporte de telemetria. O servidor de telemetria deve ser especificado no ficheiro de configuração sob a forma de um **URI completo**, e não como campos separados de endereço e porto. O dispositivo deve suportar, no mínimo, os esquemas `mqtt://` e `ws://`, selecionáveis exclusivamente por alteração do ficheiro de configuração, sem recompilação. A escolha de transporte e as suas implicações (sobrecarga por mensagem, comportamento em caso de falha, custo energético) devem ser discutidas na documentação.

Formato de dados. Todas as estruturas de dados trocadas com o exterior (mensagens de telemetria, registos no cartão microSD e valores das características GATT proprietárias) devem incluir um **campo de versão de formato**. O ficheiro de configuração JSON deve igualmente incluir um campo `config_version`.

Acesso à configuração. Os módulos não devem copiar valores de configuração para variáveis locais no arranque. O acesso aos parâmetros deve ser feito através da interface pública do módulo de configuração.

Decisões de arquitetura. O repositório deve conter um diretório `docs/adr/` com registos de decisão de arquitetura (*Architecture Decision Records*) datados, documentando as escolhas estruturais tomadas e as alternativas consideradas. Todas as decisões arquiteturais devem ser documentadas desta forma, exigindo-se um número mínimo de cinco registos.

Demonstração. Deve ser preparado um *setup* que permita visualizar em tempo real o valor das medidas que estão a ser publicadas. Isso pode ser feito recorrendo a qualquer *software open source* ou ferramenta que permita explorar e visualizar os dados em tempo real (por exemplo, MQTT Tiles, entre outras). A leitura das características BLE deve ser demonstrável com uma aplicação genérica de exploração BLE (nRF Connect, LightBlue ou equivalente).

2.2 Avaliação

A avaliação deste trabalho inclui tanto o *software*/documentação, entregue através de um repositório *GitHub*, como uma apresentação/demonstração/discussão oral a realizar em aula.

A componente de *software*/documentação será avaliada com ênfase em quatro aspetos principais:

1. Cumprimento dos requisitos funcionais/não funcionais;
2. Conceção técnica e implementação;
3. Compreensão conceptual e documentação;
4. Inovação/complexidade.

Pretende-se avaliar se as funcionalidades nucleares estão corretamente implementadas, de forma eficiente e segura, e se o ecossistema IoT produzido responde aos requisitos. Serão ainda avaliadas a qualidade do código e a adoção de uma arquitetura modular. Neste projeto, deve ser demonstrada uma integração sólida dos diferentes componentes, com decisões de *design* devidamente fundamentadas.

A documentação fornecida deve ser completa e clara. A demonstração de inovação e a capacidade de resolver problemas complexos serão valorizadas de forma distinta.

A apresentação oral avaliará a capacidade do grupo para descrever e demonstrar a solução de forma eficaz a um público generalista. Esta apresentação analisará a clareza do desenho do sistema, a demonstração prática, a capacidade de resposta a questões e o grau de domínio da solução (documentação e código apresentados).

2.2.1 Componentes da avaliação do Projeto

- Progresso da implementação no *GitHub*: 25%
- Defesa: 75%
  - Documentação entregue (relatório, *source code*, ...): 25%
  - Discussão (apresentação oral, demonstração, resposta a perguntas): 50%

2.3 Grupo

O trabalho será realizado em grupos de dois estudantes, com contributo equitativo para o resultado final. Ambos os estudantes têm de participar ativamente na apresentação/defesa oral do projeto. **A não participação na defesa determina uma classificação nula nessa componente**.

3 Registo de progresso semanal

O repositório do projeto deverá possuir um `README.md` com uma secção que constitui o registo de progresso do projeto e é objeto de avaliação na componente *Progresso da implementação no GitHub*.

3.1 Regras

1. **Periodicidade.** Deve ser acrescentada uma entrada por semana, nas seis semanas de duração do projeto. A entrada da semana N deve ser submetida até ao final de sexta-feira dessa semana para ser considerado. Se não for, considera-se o progresso nulo nessa semana.
2. **Commit próprio.** A entrada é submetida num *commit* dedicado, cuja mensagem segue o formato `log: semana N`. Esse *commit* deve ser marcado com uma *tag* `semana-N`.

3. **Registo apenas acrescentado.** As entradas de semanas anteriores não podem ser alteradas nem reescritas. O histórico do repositório permite verificar o cumprimento desta regra e a sua violação é penalizada.
4. **Entradas em falta.** A ausência de uma entrada é registada como tal. Não é aceite a submissão retroativa de várias semanas no mesmo *commit*.
5. **Dimensão.** O registo semanal deve ser compacto e focado, sendo um instrumento de acompanhamento, não um relatório.

3.2 Tabela de estado dos requisitos

A tabela dos requisitos é mantida atualizada ao longo do projeto, substituindo a versão anterior em cada entrada semanal. Os estados admissíveis são `não iniciado`, `em curso`, `funcional` e `validado`. Um requisito só pode ser declarado `validado` quando existir evidência da sua verificação, identificada na coluna respetiva e anexada na pasta `docs/evidences/`.

Req.	Descrição abreviada	Estado	Evidência de validação
RF1	Aquisição periódica DHT20	não iniciado	
RF2	Visualização no TFT	não iniciado	
...	...	...	
RF11	Estratégia de redução de consumo	não iniciado	
RNF1	Distribuição por tarefas, sem bloqueio mútuo	não iniciado	
RNF2	Tolerância a falhas de rede	não iniciado	
...	...	...	
RNF7	Interfaces públicas e independência entre módulos	não iniciado	

3.3 Modelo de entrada semanal

Copiar e preencher o bloco seguinte em cada semana, acrescentando-o ao final desta secção.

```
### Semana N (dd/mm a dd/mm)

**Requisitos trabalhados:** RFx, RFy

**Trabalho realizado.** Duas a quatro frases descrevendo o que foi concluí
   do. Identificar
os módulos criados ou alterados pelo nome dos ficheiros.

**Decisões tomadas.** Decisões de estrutura ou de interface tomadas nesta
   semana, com
referência ao ADR correspondente em `docs/adr/`. Indicar `nenhuma` quando
   aplicável.

**Dificuldades.** Problemas encontrados, incluindo os que ficaram sem
   resolução. Indicar,
para cada um, se está resolvido, contornado ou pendente. Esta alínea não
   deve ficar vazia
```

```
numa semana de trabalho efetivo.

**Verificações efetuadas.** O que foi testado e como. Quando existirem
   valores medidos
(tempos, consumos, contagens, ocupação de pilha), indicá-los com as condiç
   ões de medição.

**Divisão de trabalho.** O que foi feito por cada elemento do grupo,
   identificado pelo
número mecanográfico.

**Utilização de ferramentas de geração assistida.** Identificar as
   ferramentas usadas e
em que partes do trabalho, incluindo geração de código, depuração,
   documentação e
pesquisa. A utilização destas ferramentas é permitida; a sua omissão
   constitui falta de
integridade académica. Indicar `nenhuma` quando aplicável.

**Trabalho previsto para a semana seguinte.** Duas ou três linhas.
```

3.4 Exemplo de entrada preenchida

3.4.1 Semana 2 (06/10 a 12/10)

Requisitos trabalhados: RF1, RF4, RF7

Trabalho realizado. Implementada a tarefa de aquisição do DHT20 com periodicidade lida do ficheiro de configuração (`main/sensor_task.c`). Criado o módulo de configuração com a interface `config_get()` sobre leitura do JSON no cartão (`components/config/`). Escrita de registos em CSV no microSD operacional, com o barramento SPI partilhado com o TFT protegido por mutex (`components/storage/`).

Decisões tomadas. O acesso ao SPI passa por um mutex único em vez de uma tarefa árbitro dedicada, por simplicidade e por o TFT e o cartão não terem requisitos temporais concorrentes nesta fase. Registado em `docs/adr/0003-spi-mutex.md`.

Dificuldades. A montagem do sistema de ficheiros falhava de forma intermitente no arranque. A causa era a inicialização do cartão antes de o TFT ter terminado a sua sequência de arranque, ambos no mesmo barramento. Resolvido serializando a inicialização. Permanece pendente a decisão sobre o que fazer quando o cartão não é detetado: por agora o sistema arranca com a configuração por omissão compilada, o que contraria o RF7.

Verificações efetuadas. Aquisição verificada a 1 s, 60 s e 3600 s por inspeção dos tempos registados no CSV, com desvio inferior a 20 ms no período de 1 s. Removido o cartão com o sistema em funcionamento: a aquisição mantém-se e o erro é sinalizado no LED.

Divisão de trabalho. 112233: módulo de configuração e leitura do JSON. 445566: tarefa de aquisição e escrita no cartão. Depuração do conflito no SPI feita em conjunto.

Utilização de ferramentas de geração assistida. Consulta a um assistente para esclarecer a sequência de inicialização do `esp_vfs_fat_sdspi_mount` e para explicar mensagens de erro do driver SPI. O código dos módulos foi escrito pelo grupo.

Trabalho previsto para a semana seguinte. Cliente de telemetria com reconexão. Primeira versão do MAP server com o Sensing Service.